

Front-End Developer Interview Q&A Guide

HTML5 • Modern CSS • Tailwind CSS • Bootstrap • JavaScript (ES6+) • React

6 Topics | 300 Questions with Answers | 50+ per Topic

Topic	Basic	Intermediate	Advanced	Total
HTML5	15	20	15	50
CSS (Modern / Latest)	15	20	15	50
Tailwind CSS	15	20	15	50
Bootstrap	15	20	15	50
JavaScript (ES6+)	15	20	15	50
React	15	20	15	50

Table of Contents

1. HTML5 (50 questions)
2. CSS (Modern / Latest) (50 questions)
3. Tailwind CSS (50 questions)
4. Bootstrap (50 questions)
5. JavaScript (ES6+) (50 questions)
6. React (50 questions)

HTML5

Basic Level

Q1. What is HTML5 and how is it different from HTML4?

A: HTML5 is the latest major version of HTML, adding semantic elements, native audio/video, Canvas, form validation, and APIs like Storage and Geolocation, without needing plugins like Flash. It also relaxed strict XHTML syntax rules and defined explicit parsing/error-handling behavior.

Q2. What are semantic elements in HTML5? Give examples.

A: Semantic elements describe their meaning to both browser and developer, e.g. `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`, `<figure>`. They improve accessibility, SEO, and code readability compared to generic `<div>/<span>`.

Q3. Explain the purpose of the `<!DOCTYPE html>` declaration.

A: It tells the browser to render the page in standards mode using the HTML5 spec, avoiding quirks mode which mimics old, inconsistent browser behavior.

Q4. What is the difference between `<div>` and `<section>`?

A: `<div>` is a generic, non-semantic container used purely for styling/scripting hooks. `<section>` represents a thematic grouping of content that typically has a heading and conveys meaning to assistive technology and outline algorithms.

Q5. What are void (self-closing) elements in HTML5?

A: Elements that cannot have children and don't need a closing tag, e.g. `<br>`, `<img>`, `<input>`, `<hr>`, `<meta>`, `<link>`. In HTML5 syntax the trailing slash (`<br/>`) is optional.

Q6. Explain the difference between `<b>` and `<strong>`, `<i>` and `<em>`.

A: `<b>` and `<i>` are purely presentational (bold/italic with no extra meaning), while `<strong>` conveys strong importance and `<em>` conveys stress emphasis, both of which are announced differently by screen readers.

Q7. What is the purpose of the `alt` attribute in `<img>`?

A: It provides alternative text describing the image for screen readers, when images fail to load, and for SEO; it is required for accessibility compliance.

Q8. What are `data-*` attributes used for?

A: They let you store custom, private data on HTML elements (e.g. `data-user-id="42"`) that JavaScript can read via `dataset`, without polluting class names or invalid attributes.

Q9. Explain the difference between HTML attributes and DOM properties.

A: Attributes are defined in the HTML markup and are initial/string values; properties are the corresponding live JavaScript object values on the DOM node, which can differ from the attribute after user interaction (e.g. `input.value` vs the `value` attribute).

Q10. What is the purpose of the `meta viewport` tag?

A: It controls how a page is scaled and sized on mobile browsers, e.g. `<meta name="viewport" content="width=device-width, initial-scale=1">`, enabling responsive layouts instead of a zoomed-out desktop view.

Q11. What new form input types were introduced in HTML5 (email, date, number, etc.)?

A: Types like `email`, `url`, `tel`, `number`, `range`, `date`, `time`, `datetime-local`, `month`, `week`, `color`, and `search`, which provide built-in validation and appropriate mobile keyboards/pickers.

Q12. What is the difference between `<ol>`, `<ul>`, and `<dl>`?

A: `<ol>` is an ordered (numbered) list, `<ul>` is an unordered (bulleted) list, and `<dl>` is a description list pairing terms (`<dt>`) with descriptions (`<dd>`).

Q13. Explain the role of `<header>`, `<footer>`, `<nav>`, `<main>`, `<article>`, `<aside>`.

A: `<header>` holds intro/navigation content for a page or section; `<footer>` holds closing info; `<nav>` marks primary navigation links; `<main>` marks the unique main content (one per page); `<article>` is self-contained, independently distributable content; `<aside>` holds tangential content like sidebars.

Q14. What is the difference between `id` and `class` attributes?

A: `id` must be unique per page and is used for single-element targeting (CSS/JS/anchors), while `class` can be applied to multiple elements to share styling or behavior.

Q15. What is the purpose of the `<label>` element and how does it improve accessibility?

A: `<label>` associates descriptive text with a form control (via the `for` attribute or nesting), so screen readers announce the field's purpose and clicking the label focuses/activates the control.

Intermediate Level

Q16. Explain HTML5 form validation attributes (`required`, `pattern`, `min`, `max`).

A: `required` forces a field to be filled before submission; `pattern` applies a regex the value must match; `min`/`max` constrain numeric or date ranges. The browser blocks submission and shows native error messages when constraints fail.

Q17. What is the Canvas API and how is it different from SVG?

A: Canvas is a raster, immediate-mode drawing surface controlled via JavaScript (pixels, no DOM nodes for shapes), good for games/graphics-heavy apps. SVG is a vector, retained-mode format where each shape is a DOM element, better for scalable, interactive, or accessible graphics.

Q18. What is the difference between `localStorage`, `sessionStorage`, and cookies?

A: `localStorage` persists data with no expiration across sessions (~5-10MB, JS-only, not sent to server). `sessionStorage` is scoped to one tab and cleared on tab close. Cookies are smaller (~4KB), sent with every HTTP request, and can have expiration and server-readability.

Q19. Explain the Web Storage API.

A: It provides `localStorage` and `sessionStorage` objects for storing key-value string data in the browser synchronously, accessed via `setItem`/`getItem`/`removeItem`, useful for caching UI state or small amounts of client data.

Q20. What is the purpose of the `<video>` and `<audio>` elements and their attributes?

A: They embed media natively without plugins. Key attributes include `src`, `controls`, `autoplay`, `loop`, `muted`, and `poster` (video); multiple `<source>` children allow format fallbacks for browser compatibility.

Q21. What is the `contenteditable` attribute used for?

A: Setting `contenteditable="true"` on an element makes its content directly editable by the user in the browser, useful for building rich text editors.

Q22. Explain the difference between `inline`, `block`, and `inline-block` elements.

A: Block elements start on a new line and take full available width (e.g. `div`, `p`). Inline elements flow within text and don't accept width/height (e.g. `span`, `a`). Inline-block flows inline but respects width/height/margin like a block.

Q23. What is the difference between `defer` and `async` attributes on `<script>`?

A: `async` downloads the script in parallel and executes it as soon as it's ready, potentially out of order and blocking parsing briefly. `defer` downloads in parallel but executes only after HTML parsing completes, in document order, which is usually preferred for dependent scripts.

Q24. What is the Drag and Drop API in HTML5?

A: A native API letting elements be made draggable (`draggable="true"`) and drop targets defined via `dragover/drop` event handlers, enabling file uploads or UI reordering without a JS library.

Q25. Explain the Geolocation API.

A: `navigator.geolocation.getCurrentPosition()/watchPosition()` lets a page request the user's location (with permission), returning coordinates and accuracy, used for maps, local search, etc.

Q26. What is the difference between GET and POST in HTML forms?

A: GET appends form data to the URL as a query string (visible, cacheable, limited length, idempotent), while POST sends data in the request body (not visible in the URL, no size cap, used for state-changing operations).

Q27. What is the purpose of the <template> element?

A: It holds inert, unrendered HTML markup that can be cloned and inserted into the DOM via JavaScript later, useful for reusable UI fragments without executing scripts/loading resources upfront.

Q28. Explain the sandbox attribute for <iframe> elements.

A: It restricts what embedded content can do (e.g. disabling scripts, forms, or top-level navigation) by default, with specific permissions re-enabled individually (`allow-scripts`, `allow-same-origin`, etc.) to reduce security risk.

Q29. What is the History API (pushState, replaceState) used for?

A: It lets JavaScript manipulate the browser's session history and URL without a full page reload, enabling single-page applications to have shareable, back-button-friendly URLs.

Q30. What is the purpose of ARIA roles and attributes in HTML5?

A: ARIA (Accessible Rich Internet Applications) attributes like `role`, `aria-label`, and `aria-expanded` add semantic and state information for assistive technologies when native HTML semantics aren't sufficient, especially in custom widgets.

Q31. Explain the difference between HTML entities and Unicode characters.

A: HTML entities are text codes (e.g. `&`, `©`) that represent reserved or special characters in markup, resolved by the browser at render time; Unicode characters can often be typed/inserted directly since HTML5 documents are typically UTF-8 encoded.

Q32. What is the role of the <picture> element and srcset attribute for responsive images?

A: `<picture>` lets you specify multiple `<source>` elements with different images for different conditions (viewport size, pixel density, format), while `srcset` on `<img>` lets the browser pick the best resolution automatically, improving performance.

Q33. What is the difference between rel="noopener" and rel="noreferrer"?

A: `noopener` prevents a new tab opened via `target="_blank"` from accessing `window.opener` (a security/performance fix). `noreferrer` does the same but additionally strips the `Referer` header sent to the new page.

Q34. Explain Web Workers and how they relate to HTML5.

A: Web Workers run JavaScript in a background thread separate from the main UI thread, communicating via `postMessage`, allowing CPU-intensive tasks without freezing the page — part of the broader HTML5 platform of APIs.

Q35. What is the purpose of a web app manifest file in Progressive Web Apps?

A: A JSON `manifest.json` describes the app's name, icons, start URL, display mode, and theme colors, allowing browsers to let users install the site as a home-screen/standalone app.

Advanced Level

Q36. Explain the HTML5 parsing algorithm and error handling.

A: HTML5 formally specifies tokenization and tree-construction algorithms, including explicit recovery rules for malformed markup (e.g. auto-closing tags, implied elements), so all conforming browsers parse invalid HTML the same, predictable way, unlike the ambiguous behavior in earlier HTML.

Q37. What is Shadow DOM and how does it relate to Web Components?

A: Shadow DOM attaches a hidden, encapsulated DOM subtree to an element, scoping its styles and markup so they don't leak or clash with the main document — it's one of the three core Web Components technologies.

Q38. Explain the Custom Elements API.

A: It lets developers define new HTML tags backed by a JavaScript class (`customElements.define()`), with lifecycle callbacks (`connectedCallback`, `attributeChangedCallback`, etc.) for building reusable, framework-agnostic components.

Q39. What is the difference between older HTML Imports and using ES modules for componentization?

A: HTML Imports (deprecated, Chrome-only) let you import an `.html` file containing markup/scripts/styles as a unit. Modern practice uses ES modules to import JS logic and Custom Elements/Shadow DOM to encapsulate markup and styles, which is standardized across browsers.

Q40. How does the browser rendering engine construct the DOM tree from HTML?

A: The parser tokenizes the HTML byte stream into tags/text, builds nodes, and inserts them into a tree following the parsing algorithm's insertion-mode rules, handling implied tags and error recovery, resulting in the DOM tree used for rendering.

Q41. What is the significance of using `type="module"` together with `async/defer` on scripts?

A: Module scripts are deferred by default and execute in document order once parsed, run only once even if referenced multiple times, and are strict-mode with their own scope; `async` on a module script makes it execute as soon as it and its dependencies are fetched, regardless of order.

Q42. Explain how HTML5 handles offline capabilities (Service Workers vs the deprecated AppCache).

A: AppCache used a declarative manifest file but had many caching quirks and is deprecated. Service Workers are scriptable proxies that intercept network requests, giving fine-grained programmatic control over caching, offline fallback, and background sync.

Q43. What are microdata and Schema.org markup used for in HTML5?

A: Microdata attributes (`itemscope`, `itemtype`, `itemprop`) embed structured, machine-readable data about page content (e.g. products, reviews) directly in HTML, which search engines use for rich snippets.

Q44. Explain the difference between reflow and repaint in relation to DOM changes.

A: Reflow (layout) recalculates element geometry/position when the DOM or styles affecting layout change (expensive); repaint redraws pixels without changing layout (e.g. color changes), which is cheaper. Both impact rendering performance.

Q45. What is the purpose of the `integrity` and `crossorigin` attributes on `<script>/<link>`?

A: `integrity` provides a cryptographic hash (Subresource Integrity) so the browser verifies a fetched file wasn't tampered with (e.g. on a CDN); `crossorigin` controls whether credentials are sent and whether error details are exposed for cross-origin resources.

Q46. How would you optimize HTML for accessibility (WCAG) compliance?

A: Use semantic elements, meaningful alt text, proper heading hierarchy, labeled form controls, sufficient color contrast, keyboard-operable interactive elements, ARIA only where native semantics fall short, and test with screen readers/automated audits.

Q47. Explain lazy loading of images using the loading="lazy" attribute.

A: Setting loading="lazy" on /<iframe> tells the browser to defer loading offscreen resources until the user scrolls near them, reducing initial page weight and improving load performance natively without JavaScript.

Q48. What is the Intersection Observer API and how is it used with HTML elements?

A: It asynchronously observes when a target element's visibility relative to a viewport (or ancestor) crosses a threshold, used for efficient lazy loading, infinite scroll, and animation triggers without expensive scroll-event listeners.

Q49. Explain how HTML5 forms interact with the Constraint Validation API programmatically.

A: Form elements expose methods/properties like checkValidity(), setCustomValidity(), and the validity object (ValidityState) so JavaScript can inspect or customize validation results beyond the built-in browser UI.

Q50. What are Web Components and how do they combine Custom Elements, Shadow DOM, and HTML Templates?

A: Web Components is an umbrella term for building reusable, encapsulated UI elements: Custom Elements define the tag and behavior, Shadow DOM encapsulates its internal markup/styles, and <template> provides reusable inert markup that gets cloned into the shadow root.

CSS (Modern / Latest)

Basic Level

Q1. What is the CSS box model?

A: Every element is a box composed of content, padding, border, and margin (from inside out). Total rendered size depends on these layers plus box-sizing.

Q2. Explain the difference between margin and padding.

A: Padding is space inside the border, between the border and the content, and takes the element's background with it. Margin is space outside the border, separating the element from neighboring elements, and is transparent.

Q3. What is specificity in CSS and how is it calculated?

A: Specificity determines which rule wins when multiple selectors target the same element. It's roughly ranked as inline styles > IDs > classes/attributes/pseudo-classes > element/pseudo-element selectors, with !important overriding all of it.

Q4. Explain the difference between relative, absolute, fixed, and sticky positioning.

A: relative positions an element offset from its normal position while keeping its space. absolute removes it from flow, positioning relative to the nearest positioned ancestor. fixed positions relative to the viewport, unaffected by scrolling. sticky toggles between relative and fixed based on scroll position within its container.

Q5. What is the difference between display: none and visibility: hidden?

A: display: none removes the element entirely from layout (no space reserved, not accessible). visibility: hidden hides it visually but keeps its space in the layout and it's still present in the accessibility tree unless further restricted.

Q6. Explain the CSS cascade and inheritance.

A: The cascade resolves conflicting rules using origin, specificity, and source order to decide the winning declaration. Inheritance is separate — some properties (like color, font) pass their computed value down to descendants by default unless overridden.

Q7. What are pseudo-classes and pseudo-elements? Give examples.

A: Pseudo-classes select elements based on state or position, e.g. :hover, :nth-child(), :checked. Pseudo-elements target a sub-part of an element that isn't a real DOM node, e.g. ::before, ::after, ::first-line.

Q8. What is the difference between em, rem, %, vh, and vw units?

A: em is relative to the font-size of the current element (compounding with nesting). rem is relative to the root (<html>) font-size, avoiding compounding. % is relative to a parent's corresponding property. vh/vw are relative to 1% of the viewport's height/width.

Q9. Explain the CSS Flexbox model and its main properties.

A: Flexbox lays out items in a single row or column (flex-direction), distributing space with justify-content (main axis) and align-items/align-content (cross axis), and letting items grow/shrink via flex-grow, flex-shrink, and flex-basis.

Q10. Explain the CSS Grid layout model and its main properties.

A: Grid lays items into a two-dimensional row/column structure defined by grid-template-columns/rows, with items placed via grid-column/grid-row or named grid-template-areas, and gaps controlled by the gap property.

Q11. What is the difference between inline, block, and inline-block?

A: Block elements stack vertically and fill available width; inline elements flow horizontally within text and ignore width/height; inline-block flows horizontally like inline but supports box-model sizing like block.

Q12. What are CSS combinators (descendant, child, adjacent sibling, general sibling)?

A: Descendant (A B) selects any B inside A; child (A > B) selects direct children only; adjacent sibling (A + B) selects the B immediately following A; general sibling (A ~ B) selects any B sibling after A.

Q13. What is the purpose of the box-sizing property?

A: It determines whether width/height apply to just the content (content-box, default) or include padding and border (border-box), the latter making sizing far more predictable and commonly used as a reset default.

Q14. Explain the difference between :hover, :focus, and :active.

A: :hover applies while the pointer is over an element, :focus applies while an element has keyboard/input focus, and :active applies during the moment of activation (e.g. mouse button down on a link/button).

Q15. What is a CSS reset / normalize and why is it used?

A: A reset/normalize stylesheet neutralizes or standardizes default browser styling differences (margins, font sizes, list styles) so a design starts from a predictable, consistent baseline across browsers.

Intermediate Level

Q16. What are CSS custom properties (variables) and how are they used?

A: Declared as --name: value and read with var(--name), custom properties store reusable values (colors, spacing), cascade and inherit like normal properties, and can be changed dynamically via JavaScript or media queries, unlike Sass variables.

Q17. Explain CSS Grid template areas.

A: grid-template-areas lets you name regions of a grid as a visual ASCII-art-like layout string, then assign elements to those named areas with grid-area, making complex layouts easy to read and rearrange responsively.

Q18. What is the difference between justify-content and align-items in Flexbox?

A: justify-content aligns items along the main axis (the flex-direction), while align-items aligns items along the cross axis (perpendicular to flex-direction).

Q19. Explain media queries and responsive design principles.

A: Media queries apply CSS conditionally based on characteristics like viewport width, orientation, or resolution (@media (min-width: 768px) {...}). Responsive design combines them with fluid grids/images and a mobile-first approach to adapt layouts across devices.

Q20. What are container queries and how do they differ from media queries?

A: Container queries (@container) style an element based on the size of its nearest containing element rather than the viewport, enabling components to be responsive to their own context regardless of where they're placed on the page.

Q21. Explain the difference between min-width, max-width, and clamp().

A: min-width/max-width set a floor/ceiling on an element's width. clamp(min, preferred, max) returns a fluid value that scales with a preferred expression (often using viewport units) but is clamped between the min and max, useful for fluid typography without media queries.

Q22. What is the purpose of the calc() function?

A: calc() lets you mix units and perform arithmetic directly in CSS, e.g. width: calc(100% - 40px), useful for combining fixed and relative sizing.

Q23. Explain CSS transitions vs CSS animations.

A: Transitions smoothly interpolate a property between two states triggered by a change (like `:hover`), defined with `transition-property/duration/easing`. Animations use `@keyframes` to define multiple stages and can run automatically, loop, and have more complex timing without needing a state trigger.

Q24. What are CSS transforms (translate, scale, rotate, skew)?

A: The `transform` property visually repositions/resizes/rotates/skews an element (translate moves it, scale resizes it, rotate turns it, skew distorts angles) without affecting document flow, and is GPU-accelerated for smoother performance than changing layout properties.

Q25. Explain the concept of stacking context and z-index.

A: A stacking context is a self-contained layering group created by certain properties (`position + z-index`, `opacity < 1`, `transform`, etc.); `z-index` only compares elements within the same stacking context, which is why `z-index` sometimes 'doesn't work' as expected.

Q26. What is the difference between absolute and relative units in responsive design?

A: Absolute units (px, in, cm) render as a fixed physical/pixel size regardless of context. Relative units (em, rem, %, vw/vh) scale based on parent, root, or viewport size, making layouts adapt more naturally across screen sizes and user font preferences.

Q27. Explain CSS logical properties (margin-inline, padding-block, etc.).

A: Logical properties map to flow-relative directions (inline = text direction, block = vertical stacking direction) instead of physical directions (left/right/top/bottom), so layouts automatically adapt correctly for right-to-left or vertical writing modes.

Q28. What are the `:is()` and `:where()` selectors and how do they differ?

A: Both let you group multiple selectors compactly, e.g. `:is(header, footer)`. The difference is specificity: `:is()` takes the specificity of its most specific argument, while `:where()` always has zero specificity, making it easier to override.

Q29. What is the `:has()` selector (the 'parent selector') and what problem does it solve?

A: `:has()` lets you select an element based on its descendants or following siblings matching a condition, e.g. `div:has(> img)` selects divs containing an image — something previously impossible in pure CSS.

Q30. Explain CSS Grid auto-fit vs auto-fill.

A: Both are used with `repeat()` to create as many tracks as fit a container. `auto-fill` keeps empty tracks (reserving space) if there isn't enough content, while `auto-fit` collapses empty tracks to zero width, letting existing items stretch to fill the space.

Q31. What is native CSS nesting and how does it work?

A: Native CSS nesting lets you write child/related selectors inside a parent rule block (similar to Sass), using `&` to reference the parent selector, reducing repetition without a preprocessor, now supported directly by modern browsers.

Q32. Explain the `@supports` rule and feature queries.

A: `@supports` lets you conditionally apply CSS only if the browser supports a given property/value (e.g. `@supports (display: grid) {...}`), enabling progressive enhancement and safe fallback styling.

Q33. What is the purpose of the aspect-ratio property?

A: It lets you specify a preferred width-to-height ratio (e.g. `aspect-ratio: 16 / 9`) for an element so its box maintains proportions automatically, commonly used for responsive images/videos/placeholders.

Q34. Explain the difference between object-fit and background-size.

A: `object-fit` controls how a replaced element's content (like an `<img>` or `<video>`) is resized/cropped within its own box (e.g. `cover`, `contain`). `background-size` does the equivalent for a CSS background-image on any element.

Q35. What are CSS cascade layers (@layer) and how do they help manage specificity?

A: @layer groups rules into named layers whose order determines cascade priority independently of selector specificity — rules in a later layer always beat an earlier layer regardless of specificity, making it easier to manage overrides between resets, frameworks, and custom styles.

Advanced Level

Q36. Explain how cascade layers interact with specificity and source order.

A: Layer order is evaluated before specificity: any rule in a higher-priority layer wins over a rule in a lower layer, no matter how specific the lower-layer selector is. Only within the same layer (or in unlayered styles) does normal specificity and source order apply.

Q37. What is the difference between CSS Grid subgrid and a nested grid?

A: A nested grid is an independent grid inside a grid item with its own track sizing. subgrid lets a nested grid inherit its parent's row/column tracks directly, so items in the child align to the parent's grid lines instead of defining separate tracks.

Q38. Explain how CSS scroll-snap works.

A: scroll-snap-type on a scroll container defines an axis and strictness, while scroll-snap-align on children marks snap points; as the user scrolls, the browser 'snaps' the viewport to align with the nearest marked child, useful for carousels/galleries.

Q39. How do CSS custom property fallback values work with var()?

A: var(--name, fallback) uses the fallback value if --name is not defined (or invalid at computed-value time), allowing safe defaults; the fallback itself can contain commas by wrapping in another var() or using specific syntax.

Q40. Explain modern CSS color functions such as oklch(), lab(), and color-mix().

A: oklch()/lab() define colors in perceptually uniform color spaces, producing more predictable lightness/hue interpolation and access to a wider (P3) gamut than sRGB/hex. color-mix() blends two colors programmatically in CSS, e.g. color-mix(in oklch, red 40%, blue).

Q41. What is the purpose of the :focus-visible pseudo-class?

A: It applies a focus style only when the browser determines focus should be visibly indicated (typically keyboard navigation), avoiding the often-unwanted focus ring on mouse clicks while preserving accessibility for keyboard users.

Q42. Explain container query units (cqw, cqh, cqi).

A: These units are percentages of a query container's inline/block size or width/height (e.g. 1cqw = 1% of the container's width), allowing font sizes or spacing to scale relative to the container rather than the viewport.

Q43. What is the difference between will-change and using transform for performance optimization?

A: will-change hints to the browser that a property is about to change so it can pre-optimize (e.g. promote to its own compositor layer) ahead of time. Using transform/opacity for animations is itself a performance technique since those properties can be composited without triggering layout/paint.

Q44. Explain CSS Houdini and the Paint API.

A: Houdini is a set of low-level APIs exposing parts of the CSS engine to developers. The Paint API (registerPaint) lets you write JavaScript that generates custom background/border images procedurally, extending CSS with custom rendering logic.

Q45. What are CSS scroll-driven animations (animation-timeline)?

A: animation-timeline lets a CSS animation's progress be driven by scroll position (via scroll() or view()) instead of time, enabling effects like progress bars or reveal-on-scroll animations without JavaScript scroll listeners.

Q46. Explain the difference between CSS Modules, BEM, and utility-first CSS methodologies.

A: CSS Modules scope class names to a component automatically at build time to avoid collisions. BEM is a naming convention (Block__Element--Modifier) for writing predictable, collision-resistant plain CSS. Utility-first (e.g. Tailwind) applies many small, single-purpose classes directly in markup instead of writing custom CSS per component.

Q47. What is the purpose of the @property rule for typed custom properties?

A: @property registers a custom property with a defined syntax (type), inheritance behavior, and initial value, enabling the browser to animate/interpolate it correctly (e.g. animating a custom color or angle) instead of treating it as an opaque string.

Q48. Explain how the browser computes the critical rendering path with respect to CSS.

A: CSS is render-blocking: the browser must parse all CSS to build the CSSOM before it can construct the render tree and paint, so minimizing/deferring non-critical CSS (via media attributes, critical CSS inlining) speeds up first paint.

Q49. What is the difference between content-visibility and display: none for performance?

A: display: none removes content from rendering entirely (no layout at all). content-visibility: auto lets the browser skip layout/paint for off-screen content while still reserving approximate space (via contain-intrinsic-size), improving rendering performance for long pages without losing layout position.

Q50. How do CSS custom properties interact with the cascade compared to Sass variables?

A: CSS custom properties are live, cascade and inherit like regular CSS properties, can be read/changed at runtime via JavaScript, and respond to media queries. Sass variables are resolved entirely at compile time into static values and have no runtime presence in the browser.

Tailwind CSS

Basic Level

Q1. What is Tailwind CSS and how is it different from traditional CSS frameworks like Bootstrap?

A: Tailwind is a utility-first CSS framework providing small, single-purpose classes (e.g. flex, pt-4, text-center) to compose designs directly in markup, rather than shipping pre-built components with fixed styling like Bootstrap's .btn or .card.

Q2. What is a utility-first CSS framework?

A: One where you style elements by combining many small utility classes instead of writing custom CSS rules or relying on large pre-styled components, keeping styling co-located with markup.

Q3. How do you install and configure Tailwind CSS in a project?

A: Install via npm, run the Tailwind CLI or bundler plugin (e.g. Vite plugin) to generate CSS from a config, add the @tailwind (or @import "tailwindcss" in v4) directives to your main CSS file, and point the content/config at your source files so unused classes are purged.

Q4. What is the purpose of the tailwind.config.js file?

A: It customizes the design system — theme values (colors, spacing, fonts), which files to scan for class usage, plugins, and variants — letting you tailor Tailwind's defaults to a project's design tokens.

Q5. Explain Tailwind's spacing scale (p-4, m-2, etc.).

A: Tailwind uses a consistent numeric scale (multiples of 0.25rem by default) for padding/margin/width/height utilities, so p-4 = 1rem padding, keeping spacing consistent across a project.

Q6. What are Tailwind's responsive design breakpoints (sm, md, lg, xl, 2xl)?

A: Mobile-first breakpoint prefixes applied to any utility (e.g. md:flex) that activate the style at that minimum viewport width and above; defaults are roughly sm=640px, md=768px, lg=1024px, xl=1280px, 2xl=1536px.

Q7. How do you apply hover, focus, and active states in Tailwind?

A: Prefix any utility with the state variant, e.g. hover:bg-blue-600, focus:ring-2, active:scale-95, which Tailwind compiles into the corresponding pseudo-class CSS rule.

Q8. What is the purpose of the @tailwind directives (base, components, utilities)?

A: They mark injection points in your CSS for Tailwind's generated styles: base (resets/defaults), components (reusable class patterns), and utilities (the utility classes), which Tailwind replaces at build time (v4 simplifies this to a single @import).

Q9. Explain how Tailwind handles colors and the default color palette.

A: Tailwind ships an extensive default palette (e.g. blue-500, gray-900) with numeric shade scales, applied via utilities like bg-, text-, border-, and fully customizable/extendable through the theme config.

Q10. What is the difference between inline utility classes and using @apply?

A: Inline utilities are applied directly in HTML for one-off styling. @apply lets you compose multiple utilities into a single custom CSS class (e.g. in a component stylesheet) when you need to reuse a complex combination without repeating long class lists.

Q11. How do you create responsive layouts using Tailwind's grid and flex utilities?

A: Use flex/grid base classes with grid-cols-*, gap-*, flex-row/col, combined with responsive prefixes (e.g. grid-cols-1 md:grid-cols-3) to change layout structure at different breakpoints.

Q12. What are Tailwind's typography utility classes?

A: Classes like `text-xl`, `font-bold`, `leading-relaxed`, `tracking-wide`, and `text-gray-700` control font size, weight, line-height, letter-spacing, and color; the optional `@tailwindcss/typography` plugin adds prose classes for styling rendered markdown/rich text.

Q13. Explain how to handle dark mode in Tailwind CSS.

A: Enable dark mode strategy (media or class) in config, then prefix utilities with `dark:` (e.g. `dark:bg-gray-900`), which applies either based on the OS preference or a manually toggled class on the html element.

Q14. What is the purpose of arbitrary values in Tailwind (e.g., `w-[137px]`)?

A: Square-bracket syntax lets you use any one-off CSS value not in the default scale directly in a class name, giving Tailwind's utility approach the flexibility of custom CSS without leaving the markup.

Q15. How do you customize the default theme in Tailwind?

A: Edit the theme (or `theme.extend`, to add without removing defaults) section in `tailwind.config.js` to add/override colors, spacing, fonts, breakpoints, etc., which then become available as new utility values.

Intermediate Level

Q16. What is content configuration (unused-CSS removal) and why is it important in Tailwind?

A: The `content` option tells Tailwind which files to scan for class names so it only generates CSS for classes actually used, keeping the final CSS bundle small; misconfiguring it causes missing styles or bloated output.

Q17. Explain Tailwind's JIT (Just-In-Time) compiler and its benefits.

A: JIT generates styles on-demand by scanning your source files for class usage instead of pre-generating the entire possible utility set, resulting in much faster builds, smaller dev output, and support for arbitrary values — it's the default engine since Tailwind v3.

Q18. How do you create custom utility classes/plugins in Tailwind?

A: Use the plugin API in the config (`addUtilities`, `addComponents`, `matchUtilities`) to programmatically register new class names and their CSS, or add simple ones with `@layer` utilities in your CSS.

Q19. What are variants in Tailwind and how do you enable custom variants?

A: Variants are prefixes (like `hover:`, `dark:`, `md:`) that conditionally apply a utility. Custom variants can be added via the `addVariant` plugin API or, in v4, the `@custom-variant` directive, to target things like data attributes or custom selectors.

Q20. Explain the group and peer modifiers in Tailwind.

A: `group` lets a child style itself based on a parent's state (add `group` to the parent, `group-hover:` on the child). `peer` does the same for sibling elements (add `peer` to the trigger element, `peer-checked:` on a later sibling), enabling state-based styling without JavaScript.

Q21. How do you handle animations and transitions in Tailwind CSS?

A: Use `transition`, `duration-*`, `ease-*`, and `transform` utilities for simple hover/state transitions, or `animate-*` classes (`spin`, `ping`, `pulse`, `bounce`, or custom ones defined via keyframes in the theme) for keyframe-based animations.

Q22. What is the difference between `space-x/space-y` and `gap` utilities?

A: `space-x-*/space-y-*` add margin between sibling elements in normal flow (based on direct children), while `gap-*` is a native Flexbox/Grid property that adds spacing between items in a flex/grid container, generally the more robust modern choice.

Q23. Explain Tailwind's approach to styling pseudo-elements (before/after).

A: The `before:/after:` variants apply utilities to `::before/::after` pseudo-elements, and `content-['']` (or `content-['value']`) sets the required CSS content property so the pseudo-element actually renders.

Q24. How do you use Tailwind with component-based frameworks like React or Vue?

A: Apply utility classes directly in JSX/templates via `className/class`, optionally extracting repeated combinations into reusable components, and use libraries like `clsx` or `tailwind-merge` to conditionally compose class strings cleanly.

Q25. What is the purpose of the `tailwind-merge` and `clsx/classnames` libraries?

A: `clsx/classnames` conditionally join class name strings based on boolean logic. `tailwind-merge` intelligently resolves conflicting Tailwind utility classes (e.g. keeping only the last `px-*` when two are combined) so component props don't produce broken, conflicting CSS.

Q26. Explain how Tailwind handles container queries.

A: Tailwind v4 (and the earlier `@tailwindcss/container-queries` plugin) supports `@container`-based utilities like `@sm:` and `@lg:` after marking an element with `@container`, letting children respond to their container's size rather than the viewport.

Q27. What are the official Tailwind forms and typography plugins used for?

A: `@tailwindcss/forms` resets and normalizes form control styling so they're easy to restyle with utilities. `@tailwindcss/typography` provides a prose class that applies sensible default styling to raw HTML content like blog posts or markdown output.

Q28. How do you extend Tailwind's default theme vs override it?

A: `theme.extend` in the config adds new values alongside Tailwind's defaults (e.g. an extra color), while directly editing `theme` (outside `extend`) replaces the default scale for that key entirely.

Q29. Explain the use of the important modifier (!) on Tailwind utility classes.

A: Prefixing a utility with `!` (e.g. `!text-red-500`) makes Tailwind emit that declaration with `!important`, useful for overriding third-party or higher-specificity CSS, though it should be used sparingly.

Q30. What is the difference between Tailwind's arbitrary properties and arbitrary values?

A: Arbitrary values customize an existing utility's value with brackets (e.g. `text-[22px]`). Arbitrary properties let you write an entirely custom CSS property/value pair directly as a class using bracket syntax (e.g. `[mask-type:luminance]`).

Q31. How do you implement responsive typography using Tailwind?

A: Combine responsive text size utilities (`text-base md:text-lg lg:text-xl`) or use arbitrary values with `clamp()` for fully fluid scaling, plus responsive `leading-*/tracking-*` for line-height and letter-spacing adjustments.

Q32. Explain how Tailwind CSS integrates with CSS-in-JS or CSS Modules.

A: Tailwind can coexist with CSS Modules/CSS-in-JS by using `@apply` inside a module's custom class to compose utilities, or simply applying utility class strings directly in components while reserving CSS-in-JS for dynamic, computed styles.

Q33. What is the role of PostCSS in a Tailwind CSS build pipeline?

A: PostCSS is the transformation engine Tailwind runs as a plugin on; it processes `@tailwind` directives into actual CSS, and can be combined with other PostCSS plugins like `autoprefixer` in the same pipeline.

Q34. How would you optimize Tailwind CSS bundle size for production?

A: Ensure the content/glob configuration correctly scans exactly the files using classes (so unused ones are purged), avoid dynamically constructing class name strings at runtime (which the scanner can't detect), and let the build minify the final CSS.

Q35. Explain how Tailwind handles design tokens and consistency across a design system.

A: The theme configuration centralizes design tokens (colors, spacing, font sizes, breakpoints) as the single source of truth, which every utility class then draws from, ensuring visual consistency without hardcoding values across components.

Advanced Level

Q36. How does Tailwind CSS v4's new engine (Oxide) differ from earlier versions?

A: Oxide is a new Rust-based engine that is dramatically faster at scanning and building, uses native CSS features (like cascade layers and custom properties) more directly, supports CSS-first configuration via `@theme` instead of a JS config file, and has simplified setup (a single `@import "tailwindcss"`).

Q37. Explain how Tailwind CSS integrates with native CSS custom properties in v4.

A: v4 exposes theme values as real CSS custom properties (e.g. `--color-blue-500`) generated from the `@theme` block, so they can be referenced directly in custom CSS or read/manipulated at runtime via JavaScript, unlike the purely build-time JS config of earlier versions.

Q38. What are the trade-offs of utility-first CSS versus component-based CSS in large applications?

A: Utility-first avoids CSS bloat/naming debates and keeps styles co-located and easy to delete safely, but can make markup verbose and repetitive without componentization. Component-based CSS keeps markup clean and styles reusable by name, but risks specificity conflicts, dead CSS, and a harder-to-trace mapping between class names and appearance.

Q39. How would you architect a design system using Tailwind CSS and its config file?

A: Define all design tokens (colors, spacing, typography, radii, shadows) in the theme config as the single source of truth, build reusable component abstractions (React components or `@apply`-based classes) on top of raw utilities, and enforce usage via linting/Storybook documentation.

Q40. How would you enforce consistent utility usage across a large team using Tailwind?

A: Use ESLint/Prettier plugins (`eslint-plugin-tailwindcss`, `prettier-plugin-tailwindcss`) to sort/validate classes, restrict arbitrary values via conventions or lint rules, document approved patterns in a component library, and do code review for consistency.

Q41. What is the purpose of the `theme()` function inside custom CSS with Tailwind?

A: `theme('colors.blue.500')` lets you pull a value from the Tailwind theme config directly into custom CSS (e.g. inside a media query or a property Tailwind doesn't generate a utility for), keeping values consistent with the design system.

Q42. How do you debug specificity or conflicting utility class issues in Tailwind?

A: Because utilities are mostly single-property and low/equal specificity, conflicts are usually resolved by CSS source order; use browser devtools to inspect computed styles, use `tailwind-merge` to resolve conditional class conflicts, or use the `!` modifier as a last resort.

Q43. Explain how Tailwind CSS handles RTL (right-to-left) support.

A: Tailwind provides logical utilities (e.g. `ms-4/me-4` for `margin-inline-start/end`) that automatically adapt to the document's text direction, and community/official RTL plugins can flip physical utilities, avoiding hardcoded left/right assumptions.

Q44. How would you migrate a legacy CSS codebase to Tailwind CSS incrementally?

A: Introduce Tailwind alongside existing CSS (scoping to avoid conflicts), start converting new features/components first, use `@apply` to bridge legacy class names to utilities during transition, and gradually delete old CSS as components are fully migrated.

Q45. What are the performance implications of using many arbitrary values in Tailwind?

A: Each unique arbitrary value generates its own CSS rule rather than reusing a shared scale value, which can slightly increase CSS size and reduce cache-ability of consistent design values across the project, so they're best used sparingly for true one-offs.

Q46. Explain how Tailwind's engine scans files to generate the final CSS.

A: It treats source files as plain text and uses a regex-like scanner to extract possible class-name-looking strings (without actually parsing the language), then only generates CSS for the utilities that match a real Tailwind class, which is why dynamically concatenated class names must be avoided.

Q47. How do you create a custom Tailwind plugin that adds new utilities programmatically?

A: Write a function using the `plugin()` helper that receives helpers like `addUtilities/matchUtilities/theme`, define your custom CSS rules and value mappings, then register the plugin in the `plugins` array of the config.

Q48. What is the role of headless component libraries (e.g., Headless UI, Radix) alongside Tailwind CSS?

A: They provide fully accessible, unstyled interactive component logic (focus management, keyboard nav, ARIA) so developers can style the markup entirely with Tailwind utilities, combining accessibility correctness with full design control.

Q49. Explain accessibility considerations when building UI purely with Tailwind utility classes.

A: Utilities don't automatically ensure accessibility — you still need semantic HTML, proper focus states (`focus-visible:`), sufficient color contrast in chosen palette shades, and ARIA attributes where needed; Tailwind just applies the visual styling on top of correctly structured markup.

Q50. How would you set up a scalable multi-brand theming system using Tailwind CSS variables?

A: Define brand-agnostic semantic tokens (e.g. `--color-primary`) mapped to actual color values per brand via CSS custom properties scoped to a brand class/attribute on the root, reference those tokens in the Tailwind theme config, and switch brands by toggling the root attribute/class.

Bootstrap

Basic Level

Q1. What is Bootstrap and what problems does it solve?

A: Bootstrap is a component-based CSS/JS framework providing a responsive grid, pre-styled UI components (buttons, navbars, modals), and consistent cross-browser styling, letting teams build professional-looking, responsive interfaces quickly without writing layout CSS from scratch.

Q2. Explain the Bootstrap grid system (container, row, column).

A: A `.container` (or `.container-fluid`) wraps content and centers/pads it; `.row` holds a horizontal group of up to 12 `.col-*` columns which use Flexbox to lay out responsively, adjusting width/wrap based on breakpoint classes.

Q3. What is the difference between `.container`, `.container-fluid`, and `.container-{breakpoint}`?

A: `.container` has a fixed max-width per breakpoint with responsive padding. `.container-fluid` is always 100% width. `.container-{breakpoint}` (e.g. `.container-md`) is fluid below that breakpoint and fixed-width at and above it.

Q4. What are Bootstrap's responsive breakpoints?

A: By default: xs (<576px, implicit), sm (≥576px), md (≥768px), lg (≥992px), xl (≥1200px), xxl (≥1400px), used to prefix grid/utility classes for responsive behavior.

Q5. Explain Bootstrap's utility classes for spacing (margin/padding helpers).

A: Classes like `m-3`, `p-2`, `mt-4`, `px-5` follow the pattern `{property}{sides}-{size}`, where property is m/p, sides are t/b/s/e/x/y/blank, and size is 0-5 (or auto), providing quick spacing without custom CSS.

Q6. What is the purpose of Bootstrap's Reboot / normalize CSS?

A: Reboot resets and standardizes default browser styles (margins, font rendering, box-sizing: border-box, etc.) as a consistent, opinionated baseline before Bootstrap's own component styles are applied.

Q7. How do you include Bootstrap in a project (CDN vs npm)?

A: Via CDN, link the compiled CSS/JS files (plus Popper.js) directly in HTML for quick prototyping. Via npm, install bootstrap as a dependency and import its Sass/JS into your build pipeline for customization and tree-shaking.

Q8. Explain Bootstrap's button classes and variants.

A: The base `.btn` class combined with contextual variants like `.btn-primary`, `.btn-danger`, `.btn-outline-secondary`, sizing (`.btn-lg`/`.btn-sm`), and state classes (`.disabled`) provide consistent, themeable button styling.

Q9. What are Bootstrap's form control classes?

A: `.form-control` styles inputs/textareas, `.form-select` styles dropdowns, `.form-check`/`.form-check-input` style checkboxes/radios, and `.form-label` styles labels, all designed to work together for consistent form layouts.

Q10. Explain the key differences between Bootstrap 4 and Bootstrap 5.

A: Bootstrap 5 dropped the jQuery dependency (plain JS), dropped IE11 support, switched to a Sass-based custom property system, replaced some jQuery-plugin components' `data-*` attributes with `data-bs-*` namespacing, and added RTL support and a refreshed grid/utility system.

Q11. What is the purpose of Bootstrap's navbar component?

A: It provides a responsive navigation header with branding, links, and a collapsible menu (using `.navbar-toggler`) that adapts to a hamburger menu on small screens, built with Flexbox.

Q12. Explain Bootstrap's card component and its structure.

A: A `.card` is a flexible content container with optional `.card-header`, `.card-body` (holding `.card-title`, `.card-text`), and `.card-footer`, used for displaying grouped content like product tiles or profile summaries.

Q13. What are Bootstrap's alert components used for?

A: `.alert` with contextual classes (`.alert-success`, `.alert-danger`, etc.) displays styled, dismissible feedback messages to users, often paired with `.alert-dismissible` and a close button.

Q14. Explain Bootstrap's modal component and how it works.

A: A `.modal` is a dialog overlay toggled via `data-bs-toggle="modal"` and `data-bs-target`, or the JS Modal API; it traps focus, adds a backdrop, and fires lifecycle events (`show`, `shown`, `hide`, `hidden`) for custom behavior.

Q15. What is the purpose of Bootstrap's display utility classes (`d-none`, `d-flex`, etc.)?

A: They set an element's CSS display property (`none`, `block`, `flex`, `grid`, etc.) with responsive breakpoint variants (`d-md-none`), used to show/hide or change layout mode of elements at different screen sizes.

Intermediate Level

Q16. Explain how Bootstrap's grid system uses Flexbox internally.

A: Since Bootstrap 4, `.row` is `display: flex` with negative margins to counter column gutters, and `.col-*` classes use `flex-basis/flex-grow/max-width` to determine each column's proportional width within the 12-column system.

Q17. What is the difference between Bootstrap's offset and order utility classes?

A: Offset classes (`.offset-md-2`) push a column further right by adding `margin-left`, shifting position without reordering DOM. Order classes (`.order-2`) use Flexbox's `order` property to visually reorder columns without changing markup order.

Q18. How do you customize Bootstrap's default theme using Sass variables?

A: Import Bootstrap's Sass source, override its default `$variables` (colors, spacing, breakpoints, font settings) before the `@import` of Bootstrap's core, then compile — Bootstrap's components will use your overridden values throughout.

Q19. Explain Bootstrap's data attributes API (`data-bs-toggle`, `data-bs-target`).

A: Bootstrap's JS components can be initialized declaratively without writing JavaScript, by adding `data-bs-toggle="modal"/"collapse"/"dropdown"` and `data-bs-target="#id"` attributes to trigger elements, letting Bootstrap's JS auto-wire the behavior.

Q20. What is the purpose of Bootstrap's JavaScript components and their dependency on Popper.js?

A: JS components like dropdowns, tooltips, and popovers rely on Popper.js for dynamic, viewport-aware positioning (auto-flipping to stay visible), which Bootstrap bundles/depends on for those specific components.

Q21. Explain how Bootstrap 5 removed the jQuery dependency compared to Bootstrap 4.

A: Bootstrap 5's JavaScript plugins were rewritten in vanilla ES6+ JavaScript instead of jQuery plugins, reducing bundle size and external dependencies while keeping a similar data-attribute and JS API.

Q22. What are Bootstrap's utility classes for flexbox alignment?

A: Classes like `d-flex`, `justify-content-*`, `align-items-*`, `align-self-*`, and `flex-wrap/flex-direction` let you build and align flex layouts declaratively without custom CSS.

Q23. Explain Bootstrap's carousel component and its configuration options.

A: The `.carousel` component cycles through slides with configurable interval (autoplay timing), pause-on-hover, wrap (looping), and touch/keyboard support, controlled via data attributes or the JS Carousel API.

Q24. How do you create a responsive navigation bar with a collapsible menu using Bootstrap?

A: Wrap nav links in a `.navbar` with a `.navbar-toggler` button (`data-bs-toggle="collapse"`, targeting a `.navbar-collapse` container); Bootstrap's collapse JS shows/hides the menu below a chosen breakpoint (`.navbar-expand-lg`).

Q25. What is the purpose of Bootstrap's tooltip and popover components?

A: Tooltips show brief contextual text on hover/focus of an element; popovers are similar but support a title and richer body content, both requiring JS initialization (`new bootstrap.Tooltip(el)`) since they're opt-in for performance.

Q26. Explain Bootstrap's form validation classes (`was-validated`, `is-invalid`).

A: `was-validated` (added to a `<form>` after submission) triggers Bootstrap's styled validation feedback based on native HTML5 constraint validation; `is-invalid`/`is-valid` can be applied manually to individual fields for custom/server-side validation states.

Q27. What are Bootstrap's spacing and sizing utility naming conventions (`m-{sides}-{size}`)?

A: The pattern is `{property}{sides}-{breakpoint}-{size}`: property (`m/p`), sides (`t/b/s/e/x/y` or omitted for all sides), optional responsive breakpoint infix, and size (`0–5`, `auto`), e.g. `mt-md-3` for margin-top at md breakpoint and above.

Q28. Explain the purpose of Bootstrap's accordion component.

A: The `.accordion` component groups collapsible `.accordion-item` panels where typically only one panel is open at a time (via shared `data-bs-parent`), useful for FAQs or grouped, space-saving content.

Q29. How do you use Bootstrap's grid system for equal-width vs auto-layout columns?

A: Plain `.col` classes (without a number) create equal-width columns automatically dividing available space; `.col-auto` sizes a column to its content's natural width instead of stretching.

Q30. Explain Bootstrap's color and background utility classes.

A: Classes like `.text-primary`, `.bg-success`, `.text-white`, and `.border-danger` apply theme colors to text, backgrounds, and borders using Bootstrap's contextual color palette, keeping styling consistent with the theme.

Q31. What is the purpose of Bootstrap Icons and how are they integrated?

A: Bootstrap Icons is an official, free SVG icon library designed to match Bootstrap's aesthetic, usable via an icon font, inline SVG, or as background images, integrated by linking its CSS or importing individual SVGs.

Q32. Explain how to create a custom Bootstrap build to reduce CSS bundle size.

A: Import only the specific Sass partials/components you need (instead of the full `bootstrap.scss`), or use a build tool to tree-shake unused CSS, significantly reducing the shipped bundle compared to the full default build.

Q33. What are Bootstrap's CSS custom properties (CSS variables) introduced in Bootstrap 5?

A: Bootstrap 5 exposes many design tokens (colors, spacing, fonts) as root-level and component-level CSS custom properties (e.g. `--bs-primary`), allowing runtime theming/overrides without recompiling Sass.

Q34. Explain how Bootstrap handles dark mode support in Bootstrap 5.3+.

A: Bootstrap 5.3 introduced a color-mode system using `data-bs-theme="dark"` on the `html` or a container element, which swaps a set of CSS custom properties to dark-appropriate values across all components automatically.

Q35. What is the difference between Bootstrap's row-cols utilities and manual column classes?

A: `.row-cols-*` (e.g. `.row-cols-3`) automatically sizes all direct column children equally into that many columns per row, whereas manually setting `.col-4` on each column requires specifying the width individually and consistently.

Advanced Level

Q36. How would you integrate Bootstrap with a React or Vue application without conflicting with component-based state management?

A: Use Bootstrap's CSS for layout/utilities while relying on framework-native component libraries (react-bootstrap, BootstrapVue) instead of Bootstrap's own DOM-manipulating JS plugins, since the latter can conflict with a virtual DOM's assumptions about element ownership.

Q37. Explain Bootstrap's Sass architecture (functions, variables, mixins, maps).

A: Bootstrap's Sass source is modular: functions compute derived values (like color shades), variables hold configurable defaults, mixins encapsulate reusable rule patterns (like responsive breakpoints via media-breakpoint-up), and maps (like \$theme-colors) let you define/extend groups of related values that generate corresponding utility classes.

Q38. How do you override Bootstrap's default breakpoints in a custom build?

A: Redefine the \$grid-breakpoints (and corresponding \$container-max-widths) Sass map with your custom values before importing Bootstrap's core, and the entire grid/utility/responsive system will regenerate using your breakpoints.

Q39. Explain the accessibility features built into Bootstrap components (ARIA attributes).

A: Components like modals, dropdowns, and accordions ship with appropriate ARIA roles/states (aria-expanded, aria-hidden, role="dialog"), focus trapping in modals, and keyboard interaction support, though developers still need to supply accessible labels/content.

Q40. What are the performance trade-offs of using the full Bootstrap CSS/JS bundle versus a custom build?

A: The full bundle is simple to include but ships styles/components you may never use, increasing payload size. A custom Sass build (importing only needed partials) or PurgeCSS-style tooling produces a smaller, faster-loading bundle at the cost of more setup complexity.

Q41. How would you build a responsive dashboard layout using Bootstrap's grid and utility classes?

A: Combine a fixed/collapsible sidebar navbar with a fluid .container-fluid main content area using nested .row/.col-* for widgets, flex utilities for card alignment, and responsive breakpoint classes to stack columns on smaller screens.

Q42. Explain how Bootstrap's JavaScript plugins are structured as ES modules in Bootstrap 5.

A: Each component (Modal, Tooltip, Collapse, etc.) is its own ES module class extending a shared BaseComponent, importable individually to reduce bundle size, and instantiated either via data attributes or JS (new bootstrap.Modal(el)).

Q43. What is the purpose of Bootstrap's RFS (Responsive Font Sizes) feature?

A: RFS automatically scales font sizes (and some spacing) fluidly between breakpoints using calc()/clamp()-like logic generated at build time, so large headings shrink proportionally on smaller viewports without manual media queries.

Q44. Explain how Bootstrap handles RTL (right-to-left) layouts.

A: Bootstrap 5 generates an RTL-specific CSS build using logical properties/PostCSS RTLCSS, flipping physical properties (left/right) automatically so you can serve bootstrap.rtl.min.css for right-to-left languages with the same markup.

Q45. How would you migrate a legacy Bootstrap 3/4 project to Bootstrap 5?

A: Update class names (e.g. .ml-*/.mr-* to .ms-*/.me-*, data-toggle to data-bs-toggle), remove jQuery dependency usage, replace deprecated components, verify custom Sass overrides against the new variable/map structure, and test all JS-driven components against the new vanilla JS API.

Q46. Explain the concept of Bootstrap's utility API for generating custom utilities.

A: The `$utilities` Sass map defines all utility classes (property, values, responsive/state variants) declaratively; you can add, remove, or modify entries to generate entirely custom utility classes using Bootstrap's own generation engine.

Q47. What are common pitfalls when combining Bootstrap's CSS with a custom design system?

A: Specificity conflicts between Bootstrap's component classes and custom styles, unused/duplicated CSS bloating bundle size, visual inconsistency if only partially overriding Bootstrap's defaults, and JS plugin conflicts with custom interactive components.

Q48. Explain how Bootstrap components handle keyboard navigation and focus management.

A: Interactive components like modals trap focus within themselves while open and return it to the triggering element on close; dropdowns/menus support arrow-key navigation and Escape to close, following expected ARIA authoring practices.

Q49. How would you theme Bootstrap for multiple brands using Sass maps and CSS variables together?

A: Define brand-specific `$theme-colors` Sass maps compiled into separate builds, or use Bootstrap 5's CSS custom properties to swap a set of `--bs-*` variables at runtime per brand (via a data attribute or class on the root), avoiding a full recompile for simple color-only differences.

Q50. Explain the trade-offs between Bootstrap and utility-first frameworks like Tailwind CSS for large-scale applications.

A: Bootstrap offers faster initial development with ready-made components but can require fighting its opinionated styles for a unique design and risks CSS bloat from unused components. Tailwind offers granular design control and smaller optimized output but requires more upfront work building components and can lead to verbose markup without disciplined componentization.

JavaScript (ES6+)

Basic Level

Q1. What are the key features introduced in ES6 (ES2015)?

A: let/const, arrow functions, classes, template literals, destructuring, default/rest parameters, the spread operator, Promises, modules (import/export), Map/Set, and enhanced object literals were among ES6's major additions.

Q2. Explain the difference between var, let, and const.

A: var is function-scoped, hoisted and initialized as undefined, and can be redeclared. let is block-scoped, hoisted but not initialized (temporal dead zone), and can be reassigned. const is block-scoped like let but cannot be reassigned after initialization (though object/array contents can still be mutated).

Q3. What are arrow functions and how do they differ from regular functions?

A: Arrow functions (`() => {}`) have a concise syntax, don't bind their own this/arguments/super (inheriting from the enclosing scope instead), can't be used as constructors, and have no prototype property.

Q4. Explain template literals and tagged templates.

A: Template literals use backticks to allow multi-line strings and `${expression}` interpolation. Tagged templates prefix a template literal with a function that receives the string parts and interpolated values separately, allowing custom processing (e.g. for sanitization or i18n).

Q5. What is destructuring assignment in JavaScript?

A: A syntax for unpacking values from arrays or properties from objects into distinct variables in one statement, e.g. `const {name, age} = user;` or `const [a, b] = arr;`.

Q6. What are default parameters in ES6 functions?

A: Function parameters can specify a fallback value used when the caller omits the argument or passes undefined, e.g. `function greet(name = 'Guest') {...}`.

Q7. Explain the spread and rest operators.

A: Both use `...` syntax. Spread expands an iterable into individual elements/properties (e.g. `[...arr1, ...arr2]` or `{...obj}`). Rest collects multiple arguments/remaining elements into a single array or object (e.g. `function sum(...nums) {}` or `const [first, ...rest] = arr`).

Q8. What are ES6 classes and how do they differ from prototype-based inheritance?

A: Classes are syntactic sugar over JavaScript's existing prototype-based inheritance, providing cleaner constructor/method/extends/super syntax, but under the hood methods still live on the prototype object just as with manual prototype chains.

Q9. What are JavaScript modules (import/export)?

A: ES modules let you split code into files with explicit export (named or default) and import statements, providing static analyzability, automatic strict mode, and better dependency management than older script-tag/global-variable approaches.

Q10. Explain the difference between let/const block scope and var function scope.

A: let/const are confined to the nearest enclosing `{}` block (if, for, etc.), while var ignores block boundaries and is scoped to the nearest enclosing function (or global scope), which can lead to unexpected variable leakage with var.

Q11. What are template string interpolation and multi-line strings used for?

A: Interpolation (`${expr}`) embeds dynamic values directly into a string without concatenation; multi-line support (via backticks) lets strings span multiple lines with actual line breaks preserved, useful for readable string building/templates.

Q12. What is the for...of loop and how does it differ from for...in?

A: for...of iterates over the values of an iterable (arrays, strings, Maps, Sets), while for...in iterates over the enumerable property keys of an object (including inherited ones), which is generally unsuitable for arrays.

Q13. Explain shorthand property names and computed property names in objects.

A: Shorthand lets you write `{name}` instead of `{name: name}` when the variable name matches the key. Computed property names let you use an expression as a key, e.g. `{[dynamicKey]: value}`.

Q14. What are Map and Set objects and how are they different from Object and Array?

A: Map stores key-value pairs with keys of any type (not just strings) and preserves insertion order with better performance for frequent additions/removals. Set stores unique values only. Both provide direct size and iteration methods, unlike plain Objects/Arrays.

Q15. Explain the concept of hoisting in JavaScript.

A: Variable and function declarations are conceptually 'moved' to the top of their scope during compilation; var declarations are hoisted and initialized as undefined, function declarations are fully hoisted (callable before their line), while let/const are hoisted but remain in an inaccessible temporal dead zone until their declaration line.

Intermediate Level

Q16. What are Promises and how do they solve callback hell?

A: A Promise represents the eventual result of an async operation (pending, fulfilled, or rejected) and lets you chain `.then().catch()` handlers instead of nesting callbacks, flattening deeply nested async code into readable, linear chains.

Q17. Explain async/await and how it relates to Promises.

A: async/await is syntactic sugar over Promises: an async function always returns a Promise, and await pauses execution within that function until the awaited Promise settles, letting asynchronous code read like synchronous code.

Q18. What is the Event Loop and how does JavaScript handle asynchronous operations?

A: JavaScript is single-threaded; the event loop continuously checks if the call stack is empty and, if so, pushes the next queued callback (from the task queue or microtask queue) onto the stack, allowing async operations like timers or I/O to run without blocking.

Q19. Explain the difference between microtasks and macrotasks.

A: Microtasks (Promise callbacks, `queueMicrotask`) run immediately after the current synchronous code finishes and before rendering/other tasks, and the entire microtask queue is drained before moving on. Macrotasks (`setTimeout`, `setInterval`, I/O) are scheduled for later event loop iterations, one per loop tick.

Q20. What are generators and iterators in JavaScript?

A: An iterator is an object with a `next()` method returning `{value, done}`. A generator (function* syntax) is a function that can pause/resume execution via `yield`, automatically implementing the iterator protocol, useful for lazy sequences or custom iteration logic.

Q21. Explain the Symbol primitive type and its use cases.

A: Symbol creates a unique, immutable value often used as a non-colliding object property key (e.g. for defining custom iterables via `Symbol.iterator`), avoiding accidental property name clashes.

Q22. What is the difference between == and === in JavaScript?

A: == performs type coercion before comparing values (loose equality), while === compares both value and type without coercion (strict equality), which is why === is generally recommended to avoid unexpected coercion bugs.

Q23. Explain closures and provide a practical use case.

A: A closure is a function that retains access to its lexical scope's variables even after the outer function has returned. A practical use is creating private state, e.g. a counter function that increments an internal variable inaccessible from outside.

Q24. What is the this keyword and how does its value differ in various contexts?

A: this refers to the execution context: in a regular function it depends on how the function is called (object method, plain call, new, or explicit bind/call/apply); in an arrow function it's lexically inherited from the enclosing scope; in the global scope it's the global object (or undefined in strict mode modules).

Q25. Explain call, apply, and bind methods.

A: call(thisArg, arg1, arg2...) and apply(thisArg, [args]) invoke a function immediately with a specified this and arguments (apply takes them as an array). bind(thisArg) returns a new function permanently bound to that this, callable later.

Q26. What are WeakMap and WeakSet and when would you use them?

A: They hold object keys/values with weak references, meaning entries can be garbage-collected if no other references exist, and they're not iterable. They're used for associating metadata with objects (e.g. caching, private data) without preventing memory cleanup.

Q27. Explain the concept of currying in JavaScript.

A: Currying transforms a function taking multiple arguments into a sequence of functions each taking a single argument, e.g. add(a)(b)(c), enabling partial application and more composable, reusable function logic.

Q28. What is the difference between synchronous and asynchronous code execution?

A: Synchronous code executes line by line, blocking further execution until each statement completes. Asynchronous code allows operations (like network requests) to be initiated and completed later without blocking the main thread, using callbacks, Promises, or async/await.

Q29. Explain the Proxy and Reflect objects in ES6+.

A: Proxy wraps a target object to intercept and customize fundamental operations (get, set, has, deleteProperty, etc.) via traps, useful for validation, logging, or reactivity systems. Reflect provides corresponding default implementations of those operations, often used inside Proxy traps to forward behavior.

Q30. What is optional chaining (?.) and nullish coalescing (??)?

A: Optional chaining (obj?.prop) short-circuits to undefined instead of throwing when accessing a property on null/undefined. Nullish coalescing (a ?? b) returns the right-hand value only when the left is null or undefined (unlike || which also triggers on falsy values like 0 or "").

Q31. Explain the difference between shallow copy and deep copy in JavaScript.

A: A shallow copy (e.g. spread, Object.assign) duplicates only the top-level properties, so nested objects are still shared by reference. A deep copy recursively duplicates nested structures too, so no references are shared (e.g. via structuredClone or a recursive/library function).

Q32. What are IIFEs (Immediately Invoked Function Expressions) and are they still relevant with ES6 modules?

A: An IIFE is a function defined and immediately executed, e.g. (function(){...})(); traditionally used to create private scope and avoid polluting globals. With ES6 modules providing native scoping, IIFEs are less necessary but still used for one-off scoped initialization or in non-module scripts.

Q33. Explain array methods like map, filter, reduce, and their use cases.

A: map() transforms each element into a new array of the same length. filter() returns a new array containing only elements passing a test. reduce() accumulates all elements into a single value (sum, object, etc.) using a callback and accumulator.

Q34. What is the difference between Object.freeze() and Object.seal()?

A: Object.freeze() prevents adding, removing, or modifying properties (fully immutable at the top level). Object.seal() prevents adding or removing properties but still allows modifying existing property values.

Q35. Explain Promise.all, Promise.race, Promise.allSettled, and Promise.any.

A: Promise.all resolves when all promises resolve (rejects on first rejection). Promise.race settles as soon as the first promise settles (resolve or reject). Promise.allSettled waits for all to settle and returns each result's status. Promise.any resolves as soon as any promise fulfills, rejecting only if all reject.

Advanced Level

Q36. Explain the JavaScript prototype chain and how inheritance works under the hood.

A: Every object has an internal [[Prototype]] link to another object; property lookups walk up this chain until found or the chain ends at null. Classes and constructor functions set this chain via their prototype property, which is how method inheritance works without copying methods onto each instance.

Q37. What is the difference between deep equality and reference equality for objects?

A: Reference equality (===) checks whether two variables point to the exact same object in memory. Deep equality checks whether two (possibly different) objects have equivalent structure and values recursively, requiring a custom function or library since JavaScript has no built-in deep-equality operator.

Q38. Explain how the JavaScript engine performs garbage collection.

A: Modern engines primarily use mark-and-sweep: starting from root references (global scope, call stack), the engine marks all reachable objects, then frees memory for anything unreachable; generational collectors also separate short-lived from long-lived objects for efficiency.

Q39. What are tagged template literals used for beyond styling (e.g., in libraries like styled-components)?

A: Beyond CSS-in-JS, tagged templates are used for safely building SQL queries, GraphQL queries (gql``), internationalization/translation lookups, and automatic escaping/sanitization of interpolated values, since the tag function controls how raw parts and values are combined.

Q40. Explain the concept of debouncing and throttling and how to implement them.

A: Debouncing delays invoking a function until a pause of inactivity has passed (resetting a timer on each call), useful for search-as-you-type. Throttling ensures a function runs at most once per fixed interval regardless of how often it's triggered, useful for scroll/resize handlers.

Q41. What is memoization and how would you implement it in JavaScript?

A: Memoization caches a function's results keyed by its input arguments so repeated calls with the same inputs return the cached value instead of recomputing, typically implemented with a Map or object storing serialized argument keys mapped to results.

Q42. Explain how async generators and the for await...of loop work.

A: An async generator (async function*) can yield values that are themselves Promises or awaited internally, producing an async iterable; for await...of consumes such an async iterable, awaiting each value in sequence, useful for streaming or paginated data.

Q43. What is the difference between `structuredClone()` and `JSON.parse(JSON.stringify())` for deep cloning?

A: `structuredClone()` is a native browser/Node API that deep-clones most JS values including Dates, Maps, Sets, and circular references, without losing type information. `JSON.parse(JSON.stringify())` only handles JSON-safe data, silently drops functions/undefined/Symbols, converts Dates to strings, and fails on circular references.

Q44. Explain the module resolution differences between CommonJS and ES Modules.

A: CommonJS (`require/module.exports`) resolves and loads modules synchronously at runtime, and exports are dynamically evaluated live bindings-by-value copies. ES Modules (`import/export`) are statically analyzed at parse time (enabling tree-shaking), loaded asynchronously, and export live, read-only bindings that update if the source changes.

Q45. What is top-level `await` and how does it change module execution?

A: Top-level `await` lets an ES module use `await` directly outside an `async` function at the module's top level, pausing that module's (and dependent modules') evaluation until the awaited value resolves, useful for `async` initialization like fetching config before the rest of the module runs.

Q46. Explain how JavaScript's event delegation pattern works and why it's useful.

A: Instead of attaching a listener to every child element, you attach a single listener to a common ancestor and use `event.target` (with bubbling) to determine which child triggered it, reducing memory usage and automatically handling dynamically added elements.

Q47. What is the difference between synchronous iterators and `async` iterators?

A: A synchronous iterator's `next()` returns `{value, done}` immediately. An `async` iterator's `next()` returns a Promise resolving to `{value, done}`, allowing each iteration step to represent an asynchronous operation, consumed via `for await...of`.

Q48. Explain how the JavaScript call stack and heap work together during execution.

A: The call stack tracks function execution contexts (LIFO), pushing a frame on each call and popping it on return; primitive/local variables live there temporarily. The heap is a larger, less structured memory region storing objects/closures referenced from the stack, managed by the garbage collector.

Q49. What are private class fields and methods (`#field`) introduced in modern JavaScript?

A: Prefixing a class member with `#` (e.g. `#balance`) makes it truly private — inaccessible and invisible outside the class body, unlike the convention-only underscore prefix, enforced by the language itself rather than developer discipline.

Q50. Explain how JavaScript engines optimize code using JIT compilation (e.g., V8's hidden classes).

A: Engines like V8 first interpret code, then use a JIT compiler to compile 'hot' (frequently executed) functions into optimized machine code, using techniques like hidden classes (tracking object shape for fast property access) and inline caching; code that violates assumed shapes/types triggers deoptimization back to slower paths.

React

Basic Level

Q1. What is React and what problem does it solve?

A: React is a JavaScript library for building user interfaces using a component-based, declarative approach — you describe what the UI should look like for a given state, and React efficiently updates the DOM to match, instead of manually manipulating the DOM imperatively.

Q2. What is JSX and how does it differ from HTML?

A: JSX is a syntax extension that lets you write HTML-like markup inside JavaScript, compiled (via Babel) into `React.createElement()` calls; unlike HTML it uses camelCase attributes (`className`, `onClick`), requires a single root element/fragment, and allows embedding JS expressions with `{}`.

Q3. Explain the difference between functional and class components.

A: Functional components are plain JavaScript functions that return JSX and use Hooks (`useState`, `useEffect`) for state/lifecycle; class components extend `React.Component`, manage state via `this.state`/`this.setState`, and use lifecycle methods. Functional components with Hooks are now the standard approach.

Q4. What are props in React and how are they used?

A: Props are read-only inputs passed from a parent component to a child (as JSX attributes), used to configure or pass data into a component; a component should never mutate its own props.

Q5. What is state in React and how does it differ from props?

A: State is data managed internally by a component (via `useState`/`useReducer` or `this.state`) that can change over time and triggers re-renders when updated. Props are passed in from outside and are immutable from the receiving component's perspective.

Q6. Explain the component lifecycle methods in class components.

A: Mounting: constructor, render, `componentDidMount`. Updating: render, `componentDidUpdate` (triggered by new props/state). Unmounting: `componentWillUnmount` (cleanup). These map roughly to what `useEffect` handles in functional components.

Q7. What is the Virtual DOM and how does it improve performance?

A: The Virtual DOM is an in-memory, lightweight representation of the real DOM; React diffs the new virtual tree against the previous one (reconciliation) and applies only the minimal set of real DOM mutations needed, avoiding costly full re-renders of the actual DOM.

Q8. Explain the `useState` hook and its basic usage.

A: `const [state, setState] = useState(initialValue)` declares a piece of state in a functional component; calling `setState` schedules a re-render with the updated value, and the updater can also accept a function receiving the previous state.

Q9. What is the `useEffect` hook and how does it relate to lifecycle methods?

A: `useEffect(callback, deps)` runs side effects (data fetching, subscriptions, DOM manipulation) after render; an empty dependency array runs it once (like `componentDidMount`), specific deps re-run it on change (like `componentDidUpdate`), and a returned cleanup function runs before unmount/next effect (like `componentWillUnmount`).

Q10. Explain how to handle events in React.

A: React uses camelCase synthetic event handlers (`onClick={handleClick}`) attached directly in JSX; React wraps native events in a cross-browser SyntheticEvent object and handles event delegation internally for performance.

Q11. What is conditional rendering in React?

A: Rendering different JSX based on a condition, commonly using ternaries (`condition ? <A/> : <B/>`), logical `&&` (`condition && <A/>`), or early returns/if statements inside the component function.

Q12. How do you render lists in React and why are keys important?

A: Use `array.map()` to transform data into an array of elements, giving each a unique key prop so React can efficiently track, reorder, and update individual list items across re-renders instead of re-rendering the whole list.

Q13. What is the difference between controlled and uncontrolled components?

A: A controlled component's form value is driven entirely by React state (`value + onChange`), making React the single source of truth. An uncontrolled component keeps its own internal DOM state, accessed via a ref when needed (e.g. on submit).

Q14. Explain how to pass data from child to parent components.

A: Since props flow one-way (parent to child), the parent passes a callback function as a prop to the child; the child calls that function (optionally with data) to notify the parent, which then updates its own state.

Q15. What is React.Fragment and why is it used?

A: `<React.Fragment>` (or shorthand `<>...</>`) lets a component return multiple sibling elements without adding an extra wrapping DOM node, keeping the rendered markup cleaner.

Intermediate Level

Q16. Explain the Context API and when you would use it over prop drilling.

A: Context (`createContext`, `Provider`, `useContext`) lets you share values across a component tree without manually passing props through every intermediate level. It's best for genuinely global-ish data (theme, auth user, locale) rather than as a full state-management replacement, since overuse can hurt reusability and cause broad re-renders.

Q17. What is the useReducer hook and when would you use it instead of useState?

A: `useReducer(reducer, initialState)` manages state via dispatched actions and a pure reducer function, useful when state logic is complex, involves multiple sub-values, or the next state depends on the previous one in non-trivial ways.

Q18. Explain useMemo and useCallback and how they help with performance optimization.

A: `useMemo` memoizes a computed value, recalculating only when its dependencies change, avoiding expensive recomputation. `useCallback` memoizes a function reference itself, preventing unnecessary re-creation (and downstream re-renders of memoized children) on every render.

Q19. What is the useRef hook and what are its common use cases?

A: `useRef` returns a mutable object (`{current: value}`) that persists across renders without causing re-renders when changed; common uses include accessing DOM nodes directly, storing previous values, or holding mutable instance-like data (e.g. timer IDs).

Q20. Explain custom hooks and how you would create one.

A: A custom hook is a JavaScript function (prefixed with `use`) that calls other hooks internally to encapsulate and reuse stateful logic across components, e.g. `useFetch(url)` returning `{data, loading, error}`.

Q21. What is React.memo and how does it prevent unnecessary re-renders?

A: React.memo wraps a functional component so React skips re-rendering it if its props haven't changed (shallow comparison), useful for optimizing pure components that render often with the same props.

Q22. Explain the concept of lifting state up in React.

A: When multiple sibling components need to share/sync state, you move that state to their closest common ancestor and pass it down via props (with callbacks to update it), making the ancestor the single source of truth.

Q23. What are Higher-Order Components (HOCs)?

A: A HOC is a function that takes a component and returns a new, enhanced component with extra props/behavior (e.g. withAuth(Component)), a pattern for reusing component logic, now largely superseded by hooks but still seen in some libraries.

Q24. Explain the render props pattern in React.

A: A component accepts a function as a prop (often called render or children) that it calls with some internal data/state, letting the consumer decide how to render that data — another older pattern for sharing logic, mostly replaced by hooks.

Q25. What is React Router and how does client-side routing work?

A: React Router is a library that maps URL paths to components, intercepting navigation to update the rendered UI and browser history via the History API without a full page reload, enabling single-page application navigation.

Q26. Explain error boundaries in React and how to implement them.

A: Error boundaries are class components implementing static `getDerivedStateFromError()` and/or `componentDidCatch()` that catch JavaScript errors thrown during rendering in their child tree, displaying a fallback UI instead of crashing the whole app (functional components can't yet be error boundaries directly).

Q27. What is the difference between useEffect and useLayoutEffect?

A: `useEffect` runs asynchronously after the browser has painted, suitable for most side effects. `useLayoutEffect` runs synchronously after DOM mutations but before the browser paints, used when you need to measure/mutate the DOM and avoid a visible flicker.

Q28. Explain how React's reconciliation algorithm works (diffing).

A: React compares the new element tree to the previous one using heuristics: different element types cause a full subtree replacement; elements of the same type have their attributes diffed and updated in place; list children are matched using their key prop to minimize DOM operations.

Q29. What is the role of TypeScript or prop-types in type-checking React components?

A: Both catch type errors in props/state — TypeScript does static, compile-time type checking across the whole codebase with rich tooling support, while prop-types performs runtime checks in development, warning in the console when passed props don't match the declared shape.

Q30. Explain how to handle forms and form validation in React.

A: Typically use controlled inputs bound to state via `value/onChange`, validate on change/submit (manually or with a library like `react-hook-form` or `Formik` plus a schema validator like `Yup/Zod`), and display error state conditionally based on validation results.

Q31. What is the purpose of the key prop and what happens if it's misused?

A: `key` helps React identify which items changed, were added, or removed in a list across renders. Using array index as key when the list can reorder/filter can cause React to mismatch state/DOM to the wrong item, leading to subtle bugs like incorrect input values persisting on the wrong row.

Q32. Explain code-splitting and lazy loading with React.lazy and Suspense.

A: `React.lazy(() => import('./Component'))` dynamically imports a component only when it's needed, splitting the JS bundle; wrapping it in `<Suspense fallback={...}>` lets you show a loading state while the chunk downloads, reducing initial bundle size.

Q33. What is the difference between synchronous and batched state updates in React?

A: React batches multiple `setState` calls within the same event handler (and, since React 18, in most other contexts too) into a single re-render for performance, rather than re-rendering after every individual state update synchronously.

Q34. Explain how to fetch data in React (useEffect vs dedicated data-fetching libraries).

A: With plain `useEffect`, you manually trigger a fetch, manage loading/error/data state, and handle cleanup/race conditions on unmount or dependency change. Libraries like React Query/SWR handle caching, deduplication, background refetching, and race conditions automatically, reducing boilerplate.

Q35. What are portals in React and when would you use them?

A: `ReactDOM.createPortal(child, domNode)` renders a component's children into a different DOM node outside its parent hierarchy while keeping it in the React component tree (so events/context still work), commonly used for modals, tooltips, or dropdowns that need to escape overflow/z-index constraints.

Advanced Level

Q36. Explain React Server Components and how they differ from Client Components.

A: Server Components render entirely on the server (with zero JS shipped to the client for that component), can directly access backend resources, and can't use state/effects/browser APIs. Client Components (marked 'use client') render/hydrate in the browser and support interactivity, state, and effects; the two can be composed together in frameworks like Next.js.

Q37. What is the difference between React 18's concurrent rendering and legacy rendering?

A: Concurrent rendering lets React prepare multiple versions of the UI, interrupt/pause/resume rendering work, and prioritize urgent updates (like typing) over less urgent ones (like a big list re-render), improving responsiveness, unlike the legacy synchronous, uninteruptible render model.

Q38. Explain the useTransition and useDeferredValue hooks and their use cases.

A: `useTransition` marks a state update as a low-priority 'transition', letting urgent updates (like input typing) interrupt it, useful for expensive UI updates triggered by user interaction. `useDeferredValue` lets a value 'lag behind' the latest state during heavy renders, useful for keeping an input responsive while a derived, expensive view updates a moment later.

Q39. What is the Suspense component and how does it handle async data fetching?

A: `Suspense` lets a component 'suspend' rendering while waiting for something (like a lazy-loaded component or, with supporting libraries/frameworks, async data), showing a fallback UI until the data/resource is ready, coordinating loading states declaratively instead of manual `isLoading` flags.

Q40. Explain how React's Fiber architecture improved rendering compared to the old stack reconciler.

A: Fiber restructured reconciliation into an incremental, unit-of-work-based process that can be paused, resumed, and prioritized across multiple frames, instead of the old recursive, synchronous, uninteruptible stack-based reconciler, enabling features like concurrent rendering and time slicing.

Q41. What are the differences between Redux, Context API, and libraries like Zustand or Jotai for state management?

A: Redux provides a single global store with strict unidirectional data flow, middleware, and devtools, suited for large, complex state logic but with more boilerplate. Context is built-in but not optimized for frequent updates across many consumers. Zustand/Jotai offer lighter-weight, hook-based global/atomic state with less boilerplate and finer-grained re-render control than Context.

Q42. Explain how you would optimize a large React application for performance (memoization, virtualization, code-splitting).

A: Techniques include memoizing expensive components/values (React.memo, useMemo/useCallback), virtualizing long lists (rendering only visible rows via libraries like react-window), code-splitting routes/components with lazy/Suspense, avoiding unnecessary context re-renders, and profiling with React DevTools to find actual bottlenecks before optimizing.

Q43. What is hydration in the context of server-side rendering with React?

A: Hydration is the process where React attaches event listeners and reconciles its component tree with server-rendered static HTML already in the DOM, making the page interactive without re-rendering the markup from scratch.

Q44. Explain the difference between SSR, SSG, and ISR in relation to React rendering (e.g., in Next.js).

A: SSR (Server-Side Rendering) generates HTML per request on the server. SSG (Static Site Generation) pre-renders HTML at build time for reuse on every request. ISR (Incremental Static Regeneration) serves static pages but regenerates them in the background after a specified interval, combining SSG's speed with fresher data.

Q45. What are React's rules of hooks and why must hooks be called in the same order on every render?

A: Hooks must only be called at the top level (not inside loops/conditions/nested functions) and only from React function components/custom hooks, because React tracks hook state by call order in a linked list per component instance — inconsistent order would misalign state between renders.

Q46. Explain how you would test React components using tools like Jest and React Testing Library.

A: Jest provides the test runner/assertions, while React Testing Library renders components into a virtual DOM and encourages querying/interacting with them the way a user would (by role/text/label) rather than testing implementation details, combined with user-event for simulating realistic interactions.

Q47. What is the useSyncExternalStore hook used for?

A: It lets components safely subscribe to external (non-React) state stores while remaining compatible with concurrent rendering, ensuring consistent, tear-free reads of external state — commonly used internally by state-management libraries rather than directly by app code.

Q48. Explain how React handles reconciliation for keyed lists versus non-keyed lists.

A: With stable, unique keys, React matches elements between renders by key to determine which were added, removed, or moved, updating the DOM minimally and preserving component state correctly. Without keys (or with unstable ones like array index), React falls back to positional comparison, which can cause incorrect state reuse or unnecessary re-renders when the list changes.

Q49. What are common pitfalls with useEffect dependencies and stale closures?

A: Omitting a dependency used inside the effect can cause it to reference outdated ('stale') values from the render it was created in; overspecifying dependencies can cause effects to re-run too often. Solutions include following the exhaustive-deps lint rule, using functional state updates, or restructuring logic with useRef/useReducer where appropriate.

Q50. Explain how you would architect a scalable folder/component structure for a large-scale React application.

A: Common approaches include feature-based folders (grouping components, hooks, state, and tests by domain feature rather than by file type), a clear separation of presentational vs container/logic components, shared UI component libraries, colocated tests/styles, and consistent conventions for state management boundaries between local, feature, and global state.